

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY
PER LE TECNOLOGIE DELLA INFORMAZIONE E DELLA COMUNICAZIONE
DEL PIEMONTE

Sede legale: Torino, Piazza Carlo Felice 18

DENOMINAZIONE CORSO : **Tecnico superiore per la digitalizzazione dei processi con
soluzioni Artificial Intelligence based - DATA ANALYST E AI SPECIALIST**

Process Mining e CRM: Analisi Data-Driven per l'Ottimizzazione dei Flussi Aziendali

Candidato
SCUDERI FRANCESCO

PROPOSTA DI PROJECT WORK

Titolo del Project Work:

Process Mining e CRM: Analisi Data-Driven per l'Ottimizzazione dei Flussi Aziendali

Descrizione dell'Argomento:

Il progetto ha l'obiettivo di applicare la metodologia del Process Mining ai dati aziendali per analizzare e ottimizzare i processi operativi. Partendo dai dati storici presenti nel CRM, il lavoro si focalizza sulla ricostruzione automatica dei flussi reali tramite algoritmi di discovery. L'elaborazione permette di confrontare i processi teorici con quelli effettivi, identificando colli di bottiglia, deviazioni e inefficienze. Il risultato finale è un'analisi quantitativa che fornisce all'azienda strumenti concreti per il monitoraggio delle performance e il decision making strategico.

Indice

INTRODUZIONE	5
1 TECNOLOGIE E ARCHITETTURA DATI	9
1.1 Fondamenti di Process Mining	9
1.2 Stack tecnologico e librerie utilizzate	10
1.2.1 Linguaggio e Core (Python, PM4Py)	10
1.2.2 Backend e Gestione Asincrona (FastAPI, Redis, Celery)	10
1.2.3 Storage, Analisi e Governance (DuckDB, Polars, Pandera)	11
1.2.4 Frontend Dashboard (Streamlit)	12
1.2.5 Gestione Dipendenze e Containerizzazione (Poetry, Docker)	12
2 PIPELINE ETL E QUALITÀ DEL DATO	14
2.1 Estrazione dati e resilienza (HubSpot API e Tenacity)	14
2.2 Trasformazione e appiattimento dei log (Flattening)	16
2.3 Data Quality e validazione tramite Pandera	17
2.4 Implementazione della Privacy by Design (GDPR Compliance)	18
2.4.1 Pseudonimizzazione tramite hashing crittografico	18
2.4.2 Data Retention e Audit Logging	20
3 PROCESS DISCOVERY E CONFORMANCE CHECKING	21
3.1 Elaborazione dei modelli di processo	21
3.1.1 Generazione del Directly-Follows Graph (DFG)	21
3.1.2 Algoritmi avanzati (Alpha Miner e Heuristics Miner)	22
3.2 Analisi delle varianti di processo	23
3.3 Conformance Checking e calcolo di Fitness e Precision	24
3.4 Calcolo dei Process KPI e individuazione dei colli di bottiglia	25
4 ANALISI PREDITTIVA E INTEGRAZIONE AVANZATA	27
4.1 Feature Engineering sui log di processo	27
4.2 Modelli predittivi per il rischio e il churn	28
4.3 Meccanismi di Reverse ETL	29

4.3.1	Sincronizzazione dei KPI e degli score su HubSpot	29
4.3.2	Generazione di alert proattivi e workflow trigger	30
5	PRESENTAZIONE DEI RISULTATI E UX	31
5.1	Progettazione della User Interface (Streamlit)	31
5.2	Visualizzazione interattiva dei grafi e dei KPI	32
5.3	Dashboard per il monitoraggio predittivo e della qualità dei dati . . .	33
6	CONCLUSIONI E SVILUPPI FUTURI	35
6.1	Valutazione dei risultati raggiunti e impatto aziendale	35
6.2	Analisi delle criticità riscontrate	35
6.3	Considerazioni lavorative e prospettive di sviluppo futuro	36
	BIBLIOGRAFIA E SITOGRAFIA	37

INTRODUZIONE

Contesto aziendale e obiettivi

Nell'attuale panorama economico, la digitalizzazione e l'automazione dei processi non rappresentano più un mero vantaggio competitivo, bensì un prerequisito tecnologico fondamentale per garantire la sopravvivenza e la scalabilità aziendale. Le moderne organizzazioni fanno ampio affidamento su sistemi di *Customer Relationship Management* (CRM), come HubSpot, per tracciare le interazioni con la clientela, gestire i contatti e monitorare il ciclo di vita delle opportunità commerciali (Lead-to-Cash).

Tuttavia, all'interno dei contesti aziendali emerge frequentemente un divario significativo tra i processi "ideali", così come sono stati formalmente disegnati a livello manageriale, e i processi "reali", ovvero l'esatta sequenza di azioni quotidianamente eseguite dagli operatori sul campo. I dati inseriti all'interno del CRM, per quanto strutturati e abbondanti, tendono a fornire una fotografia statica dello stato di un *deal*: è possibile determinare con precisione se un'opportunità commerciale è stata vinta o persa, ma risulta complesso, se non impossibile tramite i classici strumenti di *Business Intelligence*, comprendere le dinamiche sottostanti. Raramente i sistemi nativi chiariscono *come* si è arrivati a un determinato risultato, calcolano il tempo di attraversamento effettivo tra le fasi o evidenziano la presenza di deviazioni non previste (es. ritorni a fasi precedenti o salti di procedure di qualificazione).

L'obiettivo primario del presente progetto di tesi è colmare questa lacuna analitica, applicando la disciplina del *Process Mining* ai dati transazionali estratti dal CRM aziendale. Il lavoro si è concentrato sullo sviluppo architetturale e logico di un **Motore di Process Intelligence**, ovvero una piattaforma software capace di estrarre la cronologia esatta dei cambi di fase direttamente dalle API di HubSpot¹, validare la qualità e l'integrità del dato e applicare algoritmi di *Process Discovery* per ricostruire automaticamente i veri flussi di lavoro aziendali. In questo modo, si fornisce all'organizzazione uno strumento di *Decision Support System* per passare da un approccio decisionale basato su intuizioni ed euristiche a uno strettamente *Data-Driven*.

¹HubSpot, Inc., [2024](#).

Analisi dei fabbisogni

L'esigenza di ingegnerizzare un sistema di analisi di processo è emersa da una fase preliminare di *Requirement Analysis*, che ha evidenziato una serie di criticità nella gestione corrente dei flussi informativi e commerciali. Nello specifico, sono stati individuati i seguenti bisogni:

- **Visibilità limitata dell'efficienza operativa:** Mentre il management possiede un'ottima visibilità sui KPI di output (es. tasso di conversione, fatturato generato), mancano all'appello metriche avanzate relative all'efficienza interna, quali il *Lead Time* (tempo totale di attraversamento di una pratica) e il *Cycle Time* (tempo effettivo di lavorazione).
- **Identificazione dei colli di bottiglia (*Bottlenecks*):** All'interno di iter commerciali per deal "su misura" o ad alta complessità, risulta complesso individuare in modo sistematico le fasi specifiche in cui le pratiche tendono a ristagnare, generando ritardi nel *time-to-market*.
- **Conformità operativa (*Conformance*):** Vi è la necessità di misurare lo scostamento tra le regole aziendali e la realtà operativa. È imperativo verificare se gli operatori seguano le procedure standard o se applichino *workaround* non documentati (es. spostamento repentino di un deal dalla prima all'ultima fase) che compromettono la *Data Quality* generale del CRM.
- **Gestione di grandi volumi di dati e UX:** Per garantire l'adozione del sistema da parte dell'utente finale, è richiesto lo sviluppo di un'architettura software asincrona. L'elaborazione degli algoritmi matematici sui grafi non deve bloccare l'interfaccia utente (UI), garantendo tempi di reattività minimi e una *User Experience* ottimale.

Metodologia e strumenti di lavoro

La metodologia scientifica adottata per l'analisi dei dati si basa sul framework teorico del *Process Mining Manifesto*², il quale suddivide l'estrazione di valore dagli *Event Log* in tre pilastri analitici:

1. **Process Discovery:** Estrazione di modelli grafici orientati a partire da dati grezzi non etichettati, senza la necessità di modelli teorici a priori. In questa fase sono impiegati algoritmi generativi per mappare l'effettivo percorso eseguito dai task nel CRM.

²Van der Aalst, 2016.

2. **Conformance Checking:** Fase di validazione che confronta il modello "as-is" ricavato dai dati con il modello procedurale "to-be" configurato nelle pipeline di HubSpot, al fine di tracciare variazioni e misurarne l'incidenza.
3. **Enhancement:** L'impiego delle evidenze raccolte per arricchire, estendere o ottimizzare il modello esistente, integrando funzionalità di intelligenza artificiale o analisi predittiva.

Sebbene il framework teorico originale contempli l'Enhancement come ottimizzazione diretta del software monitorato, nel perimetro di questo lavoro di tesi l'Enhancement viene declinato in un'ottica *Data-Driven*. I risultati del mining vengono tradotti in raccomandazioni strategiche, calcolo di KPI avanzati, *Predictive Scoring* dei deal e procedure di *Reverse ETL* (estrazione, trasformazione e ricaricamento dei dati arricchiti all'interno del CRM nativo).

Per l'implementazione pratica è stata scelta una metodologia di sviluppo di tipo *Agile*. Il ciclo di vita del software è stato gestito tramite iterazioni incrementali focalizzate dapprima sulla solidità della pipeline di *Data Engineering* (Backend ed ETL), per poi spostarsi verso l'analisi matematica (Mining) ed infine verso l'integrazione visiva (Frontend) e l'applicazione di logiche di *Machine Learning*.

Risultati implementati

A conclusione dei cicli di sviluppo, il progetto ha portato alla realizzazione di una piattaforma software *End-to-End* completa, caratterizzata dai seguenti traguardi tecnici:

Architettura Backend ed Estrazione (ETL)

È stata progettata un'architettura a microservizi basata sul framework **FastAPI**, accoppiata a **Celery** e **Redis** per l'esecuzione asincrona dei task più onerosi. La pipeline ETL (Extract, Transform, Load) è stata ottimizzata per interrogare HubSpot aggirando i rate limit, paginando le chiamate e convertendo i complessi file JSON in *Event Log* relazionali tramite la libreria ad altissime prestazioni **Polars**. È stata prestata massima attenzione al regolamento GDPR³ integrando un modulo di *Privacy by Design* capace di pseudonimizzare automaticamente i dati sensibili (come le email degli operatori) tramite hashing crittografico, applicando contestualmente logiche di *Data Retention*.

³Unione Europea, 2016.

Process Mining Engine e Data Quality

Il cuore analitico del sistema si basa sulla libreria Python **PM4Py**⁴, attraverso la quale sono stati generati *Directly-Follows Graph* (DFG) per esplorare visivamente il flusso commerciale. Il sistema elabora KPI complessi, quali i tempi medi di transizione e il *Rework Rate* (il tasso di attività ripetute all'interno della stessa pratica). Prima di ogni analisi, il dataset viene sottoposto a rigorosi controlli di *Data Quality* per verificarne completezza, correttezza dei tipi e assenza di anomalie temporali, avvalendosi del framework **Pandera**.

Interfaccia Utente e Analitica Predittiva

Il layer di presentazione dei dati è stato sviluppato mediante **Streamlit**, creando una Dashboard dinamica divisa per moduli funzionali. Gli utenti finali possono esplorare le metriche di performance, individuare i "colli di bottiglia" visualmente, e accedere ad una sezione dedicata agli *Insights Predittivi*. In tale modulo, appositi modelli di *Machine Learning* (es. Random Forest) processano le feature ingegnerizzate dagli event log per assegnare uno "score" di rischio o di probabilità di chiusura positiva ai deal attualmente aperti.

Reverse ETL e Integrazione HubSpot

A completamento del flusso informativo, il sistema non si limita alla visualizzazione passiva dei dati, ma li reintegra in HubSpot. I KPI calcolati e gli score predittivi vengono riscritti nei record originali del CRM attraverso tecniche di **Reverse ETL**, permettendo ai commerciali di visualizzare lo stato di salute del loro processo di vendita direttamente nel loro ambiente di lavoro quotidiano, abilitando di fatto la creazione di *workflow* e *alert* automatizzati.

⁴Berti, Dongen e Aalst, [2019](#).

Capitolo 1

TECNOLOGIE E ARCHITETTURA DATI

1.1 Fondamenti di Process Mining

Il Process Mining è una disciplina analitica che si colloca all'intersezione tra la *Data Science* e il *Business Process Management* (BPM). A differenza della modellazione di processo tradizionale, che si basa su interviste e documentazione cartacea (spesso soggetta a bias cognitivi o non aggiornata), il Process Mining si fonda su un approccio *fact-based*, estraendo la conoscenza direttamente dai sistemi informativi aziendali¹.

Il punto di partenza imprescindibile per qualsiasi analisi di questa natura è l'**Event Log** (registro degli eventi). Affinché i dati grezzi estratti da un CRM possano essere trasformati in un *Event Log* validabile e utilizzabile, ogni riga del dataset deve contenere rigorosamente almeno tre attributi fondamentali:

- **Case ID (Identificativo del caso):** L'identificativo univoco dell'istanza di processo tracciata end-to-end. Nel contesto di un sistema di vendita, corrisponde solitamente all'ID del *Deal* o dell'opportunità commerciale su HubSpot.
- **Activity (Attività):** L'azione o la transizione di stato eseguita all'interno dell'istanza (es. "Cambio stato da New a Qualified", oppure "Inviata proposta").
- **Timestamp (Marca temporale):** Il momento esatto in cui l'evento è stato registrato dal sistema, essenziale per calcolare i tempi di attraversamento e ordinare sequenzialmente le attività.

Nel contesto specifico di questo progetto, la sfida tecnica principale è consistita nella ricostruzione della cronologia esatta degli eventi a partire dalle API del CRM. Sistemi come HubSpot, di default, espongono tramite endpoint standard solamente lo stato *attuale*

¹Van der Aalst, [2016](#).

dell'oggetto, sovrascrivendo la storia pregressa. Per superare tale ostacolo, l'architettura implementata sfrutta in modo intensivo endpoint avanzati. Nello specifico, interrogando i parametri di tipo `propertiesWithHistory`, il sistema estrae un array cronologico di tutti i passaggi di stato di una specifica proprietà (come la *dealstage*). Tale operazione permette di tradurre un record statico di un database documentale in una traccia di processo dinamica, perfettamente compatibile con i requisiti del Process Mining.

1.2 Stack tecnologico e librerie utilizzate

Per rispondere a severi requisiti di *performance*, asincronicità e scalabilità, è stato selezionato uno stack tecnologico basato interamente sull'ecosistema Python (versione 3.12). L'architettura è stata segmentata in layer funzionali, ciascuno gestito da librerie *state-of-the-art* nel campo del *Data Engineering* e dell'Intelligenza Artificiale.

1.2.1 Linguaggio e Core (Python, PM4Py)

Python è stato scelto come linguaggio core per via della sua incontrastata leadership nell'ambito della *Data Analysis* e dell'enorme supporto della community per l'elaborazione dei dati. La modellazione matematica dei flussi è demandata interamente alla libreria **PM4Py** (Process Mining for Python) [3].

PM4Py rappresenta il vero e proprio motore di calcolo del progetto. Esso fornisce implementazioni efficienti degli algoritmi di *Discovery* (come Alpha Miner, Heuristics Miner e Inductive Miner) impiegati per la generazione del *Directly-Follows Graph* (DFG), ovvero la rappresentazione visiva dei percorsi effettuati dai *Case ID*. Inoltre, PM4Py offre moduli avanzati per il *Conformance Checking*, permettendo il calcolo vettoriale di metriche quali la *Fitness* (misura di quanto il modello riproduca accuratamente il comportamento registrato nei log) e la *Precision* (misura di quanto il modello eviti di consentire comportamenti non presenti nei log)².

1.2.2 Backend e Gestione Asincrona (FastAPI, Redis, Celery)

Una delle criticità infrastrutturali maggiori nella progettazione di pipeline analitiche è la latenza. Le API verso il CRM e l'esecuzione degli algoritmi sui grafi introducono tempi di attesa incompatibili con l'interazione in tempo reale. Un'architettura sincrona tradizionale avrebbe causato il blocco (blocking) dell'interfaccia utente durante le operazioni di *fetch* dei dati o di ricalcolo dei modelli, offrendo un'esperienza insoddisfacente.

Per ovviare a questa limitazione, l'intero *backend* è stato disaccoppiato ricorrendo ad un paradigma asincrono:

²Berti, Dongen e Aalst, 2019.

- **FastAPI:** Utilizzato per l'esposizione degli endpoint REST. Sfrutta internamente framework asincroni, garantendo altissime prestazioni nella gestione concorrente di richieste web multiple.
- **Celery:** Implementato come sistema di *task queue* distribuito. Ad esso viene demandata l'esecuzione in background (*worker*) di tutti i carichi di lavoro intensivi: dal rate limiting delle chiamate API in fase di estrazione ETL, fino alla generazione computazionale dei modelli PM4Py.
- **Redis:** Strumento *in-memory* impiegato nel duplice ruolo di *Message Broker* per Celery e di sistema di *caching*. Redis coordina i messaggi scambiati tra le API e i worker, memorizzando gli esiti delle elaborazioni.

A titolo di esempio, la gestione del routing delle code di lavoro è stata nettamente suddivisa nel codice sorgente, garantendo che i task di Data Quality non rallentino quelli di estrazione ETL, come si evince dalle configurazioni di inizializzazione di Celery:

```
celery_app.conf.update(
    task_routes={
        'app.tasks.etl_task.*': {'queue': 'etl'},
        'app.tasks.mining_task.*': {'queue': 'mining'},
        'app.tasks.dq_task.*': {'queue': 'data_quality'},
        'app.tasks.integration_task.*': {'queue': 'integration'},
    }
)
```

1.2.3 Storage, Analisi e Governance (DuckDB, Polars, Pandera)

Per la trasformazione e la manipolazione strutturale dei dati in memoria (fase di *Transformation*), si è optato per **Polars**. Sfruttando un motore scritto in Rust e basato sul formato *Apache Arrow*, Polars esegue le operazioni in modalità *multithreading* e tramite query "lazy" ottimizzate, offrendo prestazioni e un'efficienza nella gestione della memoria nettamente superiori rispetto al tradizionale approccio *single-threaded* di Pandas.

La persistenza analitica locale è stata implementata tramite **DuckDB**. Questo sistema RDBMS *in-process*, orientato al mondo OLAP (Online Analytical Processing), permette di eseguire interrogazioni SQL complesse direttamente sui file Parquet o sulle variabili in memoria generate da Polars, senza l'overhead tipico di un database server client-server come PostgreSQL. Il salvataggio dei dataframe avviene in modo diretto e integrato, riducendo drasticamente il codice boilerplate, come visibile nella funzione di gestione del database:

```
def save_event_log(df: pl.DataFrame, table_name: str = "event_log"):
    conn = get_db_connection()
    try:
        conn.execute(f"CREATE OR REPLACE TABLE {table_name} AS SELECT * FROM df")
    except Exception as e:
        logger.error(f"Errore durante il salvataggio nel DB: {e}")
        raise
    finally:
        conn.close()
```

In parallelo, le operations di *Data Governance* e *Data Quality* sono monitorate attraverso **Pandera**³. Questo framework di *schema validation* intercetta i dataframe in ingresso verificandone la conformità a specifiche rigorose: tipo di dato corretto, presenza ineludibile dei campi obbligatori, formato coerente dei timestamp e identificazione precoce di anomalie che potrebbero inficiare le analisi del Process Mining.

1.2.4 Frontend Dashboard (Streamlit)

Per l'interfaccia utente e l'esposizione interattiva degli analytics è stato selezionato **Streamlit**. Si tratta di un framework capace di convertire script Python in dashboard applicative in modo istantaneo, azzerando la complessità legata allo sviluppo frontend puro (HTML/JS/CSS). Tramite Streamlit sono state realizzate diverse aree funzionali:

- *Dashboard Processo*: Per la visualizzazione del *Sankey diagram* e dei colli di bottiglia.
- *Qualità Dati*: Per il monitoraggio in tempo reale dei punteggi di completezza e validità tramite metriche Pandera.
- *Insights Predittivi*: Per esporre le probabilità di conversione generate dai modelli algoritmici.

1.2.5 Gestione Dipendenze e Containerizzazione (Poetry, Docker)

A garanzia della riproducibilità ambientale e della stabilità infrastrutturale, la gestione dei pacchetti software è stata affidata a **Poetry**. Rispetto al tradizionale strumento pip, Poetry implementa un risolutore di dipendenze deterministico basato su *lockfile* (`poetry.lock`). Tale approccio ha evitato categoricamente l'insorgenza di conflitti di versione tra l'ecosistema web (FastAPI, Uvicorn) e l'ecosistema matematico-scientifico (PM4Py, Polars, scikit-learn).

³Bantilan, [2020](#).

Infine, l'intera applicazione è stata virtualizzata utilizzando **Docker** e **Docker Compose**. La containerizzazione isola i vari microservizi: un container dedicato ospita il server API, un secondo *container image* avvia Redis, e un terzo isola i worker asincroni di Celery, permettendo al sistema di scalare orizzontalmente con facilità in ambienti di produzione.

Capitolo 2

PIPELINE ETL E QUALITÀ DEL DATO

La fondatezza e l'affidabilità di qualsiasi analisi di *Process Mining* dipendono intrinsecamente dalla qualità e dalla strutturazione dei dati in ingresso. In ambito aziendale, i dati grezzi risiedono all'interno del CRM sotto forma di documenti non relazionali, inadatti ad essere processati direttamente dagli algoritmi matematici sui grafi. Per colmare tale divario, è stata ingegnerizzata una complessa pipeline ETL (*Extract, Transform, Load*), progettata per estrarre la storia transazionale, modellarla in un formato tabellare standardizzato e garantirne l'integrità attraverso rigide regole di validazione.

2.1 Estrazione dati e resilienza (HubSpot API e Tenacity)

La fase di estrazione (Extraction) comunica con l'ecosistema HubSpot tramite le API REST ufficiali¹. I sistemi CRM in *cloud*, tuttavia, impongono limitazioni rigorose sul numero di richieste effettuabili in un dato lasso di tempo (*Rate Limiting*), al fine di preservare la stabilità dei propri server. Quando tali limiti vengono superati, le API restituiscono un codice di errore HTTP 429 (*Too Many Requests*), che in un'architettura software naïf causerebbe il fallimento dell'intero processo di ingestione.

Per garantire la massima resilienza e *fault-tolerance* del sistema, il client HTTP sviluppato in Python è stato avvolto da meccanismi di *Exponential Backoff* e *Retry* automatici, delegati alla libreria **Tenacity**. Invece di interrompere l'esecuzione, il sistema intercetta le eccezioni di rete e i codici 429, mettendo il *worker* in pausa per un intervallo di tempo che cresce esponenzialmente ad ogni tentativo fallito, fino al raggiungimento di una soglia massima.

¹HubSpot, Inc., [2024](#).

Di seguito viene riportato lo stralcio di codice relativo al *core* del connettore HubSpot, in cui si evidenzia l'implementazione del decoratore di retry:

```
@retry(
    retry=retry_if_exception_type((requests.exceptions.RequestException, RateLimitError)),
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=4, max=60),
    reraise=True
)

def _make_request(self, method: str, endpoint: str, params: Dict = None,
                  data: Dict = None) -> Dict:

    current_time = time.time()
    if current_time - self.last_request_time < self.rate_limit_delay:
        time.sleep(self.rate_limit_delay - (current_time - self.last_request_time))

    url = f"{self.base_url}{endpoint}"

    try:
        response = requests.request(
            method=method, url=url, headers=self.headers,
            params=params, json=data, timeout=30
        )

        self.request_count += 1
        self.last_request_time = time.time()

        if response.status_code == 429:
            logger.warning(f"Rate limit raggiunto. Tentativo {self.request_count}")
            raise RateLimitError("HubSpot rate limit exceeded")

        return response.json()

    except requests.exceptions.Timeout:
        logger.error("Timeout nella richiesta a HubSpot")
        raise
```

In aggiunta alla gestione degli errori, il connettore implementa algoritmi di *paginazione* automatica tramite i token after, garantendo la capacità di scaricare iterativamente data-

set composti da decine di migliaia di *deal* senza saturare la memoria RAM del container Docker.

2.2 Trasformazione e appiattimento dei log (Flattening)

Il payload JSON restituito da HubSpot presenta una struttura documentale profondamente annidata, che descrive lo stato del *Deal* e le sue proprietà. Per estrarre gli eventi storici necessari al Process Mining, il sistema interroga l'endpoint richiedendo esplicitamente il parametro `propertiesWithHistory` per il campo `dealstage`.

La fase di trasformazione (Transformation) ha il compito di eseguire il *flattening* del JSON, disaggregando l'albero gerarchico in una serie di record piatti (*flat rows*). Ogni qualvolta il sistema rileva uno storico di cambi di fase per un singolo deal, genera *N* record distinti, corrispondenti agli *N* passaggi di stato registrati.

Questa logica di mappatura associa dinamicamente gli ID grezzi delle fasi di HubSpot (es. "appointmentscheduled") in stringhe leggibili ("Appuntamento Pianificato"). Inoltre, si occupa del parsing e della normalizzazione dei timestamp, convertendoli dallo standard ISO 8601 a oggetti `Datetime` manipolabili matematicamente. Il processo è incapsulato nella logica del mapper:

```
def map_deal_to_event_log(self, deal_data: Dict[str, Any]) -> List[Dict[str, Any]]:
    events = []
    deal_id = deal_data.get('id')
    history_data = deal_data.get('propertiesWithHistory', {})
    stage_history = history_data.get('dealstage', [])

    # Ordinamento cronologico degli eventi per singolo caso
    stage_history.sort(key=lambda x: x.get('timestamp', 0))

    for record in stage_history:
        stage_value = record.get('value', '')
        activity_name = self.stage_mapping.get(
            stage_value.lower(),
            f"Fase Sconosciuta: {stage_value}"
        )

        timestamp = self._parse_timestamp(record.get('timestamp'))
        source_id = record.get('sourceId', 'System')
```

```

event = {
    "case_id": deal_id,
    "activity": activity_name,
    "timestamp": timestamp,
    "resource": privacy_manager.hash_email(source_id),
    "amount": deal_data.get('properties', {}).get('amount', ''),
    "pipeline": deal_data.get('properties', {}).get('pipeline', '')
}
events.append(event)

return events

```

2.3 Data Quality e validazione tramite Pandera

Nelle scienze dei dati vige l'assioma "*Garbage In, Garbage Out*" (GIGO): modelli algoritmici sofisticati produrranno inevitabilmente risultati fallaci se alimentati con dati corrotti. Per prevenire questa eventualità, la pipeline ETL è stata dotata di un "casello di controllo" matematico implementato tramite **Pandera** [5], un framework avanzato per la validazione di schemi su dataframe *Polars* e *Pandas*.

La classe `EventLogSchema` definisce un contratto tipizzato a cui i dati estratti devono sottostare in modo ineludibile prima di essere persistiti nel *Data Lake* (in formato `.parquet`) e passati al motore di PM4Py. Vengono eseguiti tre livelli di validazione:

1. **Schema Validation:** Verifica della presenza di tutte le colonne obbligatorie (`case_id`, `activity`, `timestamp`) e coercizione dei tipi di dato (*type casting*).
2. **Completeness Check:** Controllo dell'assenza di valori nulli o stringhe vuote nei campi chiave, essenziali per la topologia del grafo.
3. **Consistency Check:** Analisi logico-temporale degli eventi, volta a rintracciare record duplicati o sequenze di timestamp invertite, che comporterebbero la creazione di loop impossibili nel modello di processo.

La sintassi dichiarativa di Pandera consente di definire espressioni lambda per la validazione a livello di singolo elemento, come riportato nella configurazione dello schema di progetto:

```

class EventLogSchema:
    schema = DataFrameSchema({
        "case_id": Column(str, nullable=False, checks=[

```

```

        Check(lambda x: x.str.len() > 0, element_wise=False)
    ]),
    "activity": Column(str, nullable=False, checks=[
        Check(lambda x: x.str.len() > 0, element_wise=False)
    ]),
    "timestamp": Column(pl.Datetime, nullable=False),
    "resource": Column(str, nullable=True),
    "amount": Column(str, nullable=True),
    "environment": Column(str, nullable=True)
}, strict=False)

```

Qualora l'event log non superi i controlli, il sistema intercetta l'eccezione, blocca il caricamento nel database analitico DuckDB e registra l'anomalia negli appositi file di log e sulle dashboard utente, garantendo che le analisi a valle si basino esclusivamente su dataset *gold-standard*.

2.4 Implementazione della Privacy by Design (GDPR Compliance)

L'estrazione e l'elaborazione di dati transazionali da un CRM aziendale comportano inevitabilmente il trattamento di informazioni personali, quali gli indirizzi email degli operatori commerciali, i nomi dei clienti e i dettagli di contatto. Al fine di allineare l'architettura software alle disposizioni del Regolamento Generale sulla Protezione dei Dati (GDPR) [4], il sistema è stato ingegnerizzato seguendo il principio della *Privacy by Design*.

Tale paradigma impone che la tutela della riservatezza sia integrata nativamente nell'architettura del sistema informativo fin dalle prime fasi di progettazione, e non aggiunta come modulo a posteriori. A questo scopo, è stato sviluppato un *PrivacyManager* dedicato, il quale agisce come middleware tra la fase di estrazione pura e il caricamento nel Data Lake, garantendo l'anonimizzazione dei log e il rispetto dei tempi di conservazione.

2.4.1 Pseudonimizzazione tramite hashing crittografico

Nel *Process Mining*, per calcolare i KPI di performance del personale (es. tempi medi di risoluzione per utente) e ricostruire la rete sociale delle interazioni (*Social Network Analysis*), è fondamentale mantenere traccia della risorsa che ha eseguito un'attività. Tuttavia, memorizzare l'identità in chiaro (es. "mario.rossi@azienda.com") costituisce una violazione del principio di minimizzazione dei dati.

Per risolvere questo trade-off, la pipeline ETL implementa una funzione di pseudonimizzazione basata su funzioni di *hashing* crittografico ad una via. Il sistema converte l'identificativo in chiaro in una stringa alfanumerica incomprensibile, avvalendosi dell'algoritmo MD5 rafforzato tramite l'aggiunta di un *salt* crittografico predefinito nelle variabili d'ambiente. Ciò assicura che due operatori diversi abbiano sempre hash diversi, mantenendo l'integrità statistica del log senza esporre il dato personale.

La logica di *hashing* è implementata come segue nel *core* dell'applicazione:

```
import hashlib

class PrivacyManager:
    def __init__(self):
        self.salt = settings.email_hash_salt
        self.pseudonymization_enabled = settings.pseudonymization_enabled

    def hash_email(self, email: str) -> str:
        if not email or not email.strip():
            return "Unknown"

        email_normalized = email.strip().lower()
        hash_input = f"{email_normalized}{self.salt}"
        hash_value = hashlib.md5(hash_input.encode('utf-8')).hexdigest()

        return f"User_{hash_value[:16]}"
```

Questa funzione viene poi mappata vettorialmente sull'intero dataframe Polars durante la fase di trasformazione, garantendo alte prestazioni anche su milioni di righe:

```
def _apply_pseudonymization(self, df: pl.DataFrame) -> pl.DataFrame:
    sensitive_columns = ['resource', 'email']
    df_pseudonymized = df.clone()

    for column in sensitive_columns:
        if column in df_pseudonymized.columns:
            df_pseudonymized = df_pseudonymized.with_columns([
                pl.col(column).apply(
                    lambda x: privacy_manager.hash_email(str(x))
                ).alias(column)
            ])
    return df_pseudonymized
```

2.4.2 Data Retention e Audit Logging

Oltre alla pseudonimizzazione, la *compliance* normativa impone vincoli stringenti sulla limitazione della conservazione dei dati (Storage Limitation) e sull'*Accountability* (Responsabilizzazione).

Per rispondere al primo requisito, è stato implementato un *demone* asincrono incaricato di far rispettare le *Data Retention Policies*. Il servizio analizza periodicamente i file salvati all'interno delle directory *raw*, *staged* e *processed* del Data Lake, confrontando la marca temporale di ultima modifica (ottenuta tramite la funzione di sistema `stat().st_mtime`) con una finestra temporale massima configurabile (es. 365 giorni). Qualora un frammento di dati superi tale limite, il file viene irreversibilmente eliminato (*unlinked*) dal file system. A garanzia dell'*Accountability*, la classe `PrivacyGovernanceService` implementa un rigoroso meccanismo di *Audit Logging*. Ogni qualvolta un processo algoritmico o un utente finale richiede l'accesso a porzioni del dataset, il sistema registra una voce di log in formato JSON (`privacy_audit.log`). La traccia contiene il timestamp, la tipologia di operazione richiesta, l'identificativo del richiedente e un *flag* che indica se l'accesso ha coinvolto colonne originariamente classificate come sensibili. Questo livello di tracciabilità si rivela uno strumento indispensabile in sede di audit aziendale o certificazione ISO.

Capitolo 3

PROCESS DISCOVERY E CONFORMANCE CHECKING

L'estrazione di un *Event Log* validato e normalizzato, per quanto essenziale, costituisce solamente il prerequisito per l'applicazione delle tecniche di *Process Intelligence*. Il Capitolo 4 esplora il nucleo matematico del sistema, illustrando come i dati tabellari vengano trasformati in modelli comportamentali attraverso algoritmi di *Process Discovery*, e come tali modelli vengano successivamente valutati tramite logiche di *Conformance Checking*.

3.1 Elaborazione dei modelli di processo

L'obiettivo primario della fase di *Discovery* è sintetizzare un modello di processo a partire da un registro di eventi non etichettato, senza alcuna assunzione aprioristica su come il processo dovrebbe comportarsi [1]. Nel contesto di un CRM aziendale, ciò si traduce nella mappatura visiva e quantitativa dell'intero ciclo di vita di un *Deal*, dall'acquisizione del *Lead* fino alla chiusura del contratto (Won/Lost).

Per l'implementazione di questi algoritmi, l'architettura si affida alla libreria **PM4Py** [3], integrata all'interno del servizio di dominio *DiscoveryService*.

3.1.1 Generazione del Directly-Follows Graph (DFG)

Il modello più intuitivo e computazionalmente efficiente per l'analisi esplorativa è il *Directly-Follows Graph* (Grafo di Diretta Consecuzione). Un DFG è un grafo orientato $G = (N, E)$ dove i nodi N rappresentano le singole attività (gli stati del *Deal* su HubSpot) e gli archi diretti E rappresentano le transizioni osservate tra un'attività e la successiva. Ogni arco è pesato in base a due possibili metriche vettoriali:

- **Frequenza assoluta:** Il numero di volte in cui una specifica transizione si è verificata all'interno dell'intero log.

- **Performance (Tempo medio):** Il delta temporale medio trascorso per passare dal nodo sorgente al nodo destinazione, fondamentale per individuare i *bottleneck* (colli di bottiglia).

Nel sistema sviluppato, la generazione del DFG è incapsulata in una classe asincrona che riceve in input il dataframe Polars convertito nel formato standard di PM4Py. Di seguito è riportato lo stralcio relativo al calcolo delle frequenze e dei tempi di attraversamento:

```
import pm4py
from pm4py.objects.log.obj import EventLog

class DiscoveryService:
    def __init__(self, log: EventLog):
        self.log = log

    def generate_dfg_frequencies(self) -> dict:
        dfg, start_activities, end_activities = pm4py.discover_dfg(self.log)
        return {
            "edges": dfg,
            "start_nodes": start_activities,
            "end_nodes": end_activities
        }

    def generate_dfg_performance(self) -> dict:
        dfg_perf, start_act, end_act = pm4py.discover_performance_dfg(self.log)
        return {
            "edges_performance": dfg_perf,
            "start_nodes": start_act,
            "end_nodes": end_act
        }
```

3.1.2 Algoritmi avanzati (Alpha Miner e Heuristics Miner)

Sebbene il DFG fornisca una panoramica immediata, esso soffre di una severa limitazione analitica: non è in grado di rappresentare costrutti logici complessi come la concorrenza (task eseguiti in parallelo) o le scelte esclusive (XOR gateway). Per modellare tali dinamiche, il sistema implementa algoritmi di discovery più sofisticati, restituendo in output modelli formali noti come *Reti di Petri*.

Il primo algoritmo valutato è stato l'**Alpha Miner**. Basandosi sulla rilevazione di relazioni causali tra le attività (relazioni di sequenza, di scelta, di parallelismo e di non

connessione), l'Alpha Miner è in grado di costruire una Rete di Petri matematicamente rigorosa. Tuttavia, test empirici sui dati reali di HubSpot hanno dimostrato che l'Alpha Miner è intollerante al rumore: anche una singola anomalia (ad esempio un commerciale che salta per errore una fase e vi ritorna subito dopo) genera un modello noto in gergo come "*Spaghetti Process*", ovvero un grafo talmente denso di archi da risultare illeggibile. Per superare questo limite, il motore è stato impostato per utilizzare di default l'**Heuristics Miner**. Questo algoritmo applica un approccio probabilistico: invece di considerare ogni singola transizione come una regola ferrea, calcola una misura di dipendenza basata sulle frequenze. Tramite l'impostazione di soglie parametriche (*Dependency Threshold* e *Observation Threshold*), il sistema scarta le transizioni anomale e a bassa frequenza, restituendo il flusso *Mainstream* dell'azienda. L'implementazione prevede la calibrazione dinamica di questi parametri direttamente dalla *Dashboard* utente:

```
def discover_heuristics_net(self, dependency_thresh: float = 0.5):
    # L'Heuristics Miner tollera il rumore filtrando archi rari
    net, im, fm = pm4py.discover_petri_net_heuristics(
        self.log,
        dependency_threshold=dependency_thresh,
        and_threshold=0.65,
        loop_two_threshold=0.5
    )
    return net, im, fm
```

3.2 Analisi delle varianti di processo

Un ulteriore livello di astrazione analitica è fornito dall'analisi delle *Varianti*. In ambito Process Mining, una variante è definita come una sequenza unica e specifica di attività, dall'inizio alla fine del caso. Due *Deal* che attraversano esattamente gli stessi stati nel medesimo ordine appartengono alla stessa variante, indipendentemente dal tempo impiegato o dall'operatore coinvolto.

L'estrazione delle varianti permette al management di rispondere a una domanda cruciale: "*Qual è il percorso standard che la maggior parte delle nostre pratiche segue?*". Frequentemente, il *Teorema di Pareto* si applica anche ai processi aziendali: il 20% delle varianti copre l'80% del volume totale dei log, rappresentando il processo *Happy Path* (il percorso ideale e senza intoppi). Il restante 80% delle varianti costituisce la *Long Tail* (coda lunga), composta da eccezioni, *rework* (rilavorazioni), e iter complessi.

Il modulo di calcolo sviluppato aggrega i dati vettorialmente per estrarre la distribuzione di frequenza delle varianti. In questo modo l'interfaccia frontend è in grado di mostrare un

grafico a torta o un istogramma che evidenzia quanto il processo aziendale sia standardizzato rispetto a quanto sia frammentato in casistiche eccezionali. Una forte proliferazione di varianti a bassa frequenza è un indicatore primario di scarsa *governance* procedurale all'interno del team di vendita.

3.3 Conformance Checking e calcolo di Fitness e Precision

Se la fase di *Discovery* mira a costruire un modello a partire dai dati, il *Conformance Checking* procede nella direzione opposta: accoppia un modello di processo di riferimento (spesso il modello "To-Be" teorizzato dall'azienda o generato precedentemente) con l'Event Log attuale ("As-Is"), al fine di misurare oggettivamente le discrepanze [1].

In ambito aziendale, gli operatori commerciali dovrebbero seguire pipeline rigide imposte nel CRM. Tuttavia, deviazioni come salti di fase, ritorni a stati precedenti (*rework*) o chiusure premature senza l'opportuna qualificazione del *Lead* sono all'ordine del giorno. Il sistema valuta l'aderenza a queste regole tramite due metriche matematiche fondamentali, calcolate sulle Reti di Petri scoperte in precedenza:

- **Fitness (Replay Fitness):** Misura la porzione di comportamento registrato nel log che può essere riprodotta (o "accettata") dal modello. Un valore di Fitness pari a 1.0 (100%) indica che ogni singola traccia presente nel CRM è perfettamente contemplata dalle regole del modello.
- **Precision:** Misura la capacità del modello di non generalizzare eccessivamente. Un modello con Precisione pari a 1.0 (100%) non consente alcun comportamento che non sia stato esplicitamente osservato nel log.

Nel motore algoritmico del progetto, il calcolo di queste metriche è stato implementato ricorrendo alla tecnica dell'*Alignment-based Conformance Checking* (Allineamenti). Rispetto al più datato *Token Replay*, gli allineamenti trovano matematicamente il percorso valido più vicino a una traccia non conforme, permettendo di localizzare il punto esatto in cui l'operatore ha violato la procedura. Il servizio dedicato esegue i calcoli in background, data la loro natura intensiva a livello computazionale:

```
import pm4py
from pm4py.algo.evaluation.replay_fitness import algorithm as fitness_evaluator
from pm4py.algo.evaluation.precision import algorithm as precision_evaluator

class ConformanceService:
```

```

def __init__(self, log, net, initial_marking, final_marking):
    self.log = log
    self.net = net
    self.im = initial_marking
    self.fm = final_marking

def calculate_fitness(self) -> float:
    fitness_result = fitness_evaluator.apply(
        self.log, self.net, self.im, self.fm,
        variant=fitness_evaluator.Variants.ALIGNMENT_BASED
    )
    return fitness_result['average_trace_fitness']

def calculate_precision(self) -> float:
    precision_result = precision_evaluator.apply(
        self.log, self.net, self.im, self.fm,
        variant=precision_evaluator.Variants.ALIGN_ETCONFORMANCE
    )
    return precision_result

```

L'esposizione di tali indici nella Dashboard Streamlit offre al management un "termometro" istantaneo della standardizzazione operativa: un brusco calo della Fitness nel corso di una settimana è un forte segnale di allarme per il controllo qualità.

3.4 Calcolo dei Process KPI e individuazione dei colli di bottiglia

L'ultimo tassello analitico fornito dagli algoritmi sui grafi riguarda la dimensione temporale e prestazionale del flusso commerciale. L'individuazione empirica dei colli di bottiglia (*Bottleneck Analysis*) è un'esigenza primaria, in quanto un rallentamento in una singola fase critica impatta l'intera filiera del *Lead-to-Cash*.

Per soddisfare questa richiesta, il modulo KPIService estrae dal *Directly-Follows Graph* e dal registro degli eventi una serie di indicatori chiave di prestazione (*Key Performance Indicators*):

1. **Throughput Time (Tempo di attraversamento):** Misurato come il delta temporale totale tra il primo evento di un caso (es. *Lead Created*) e il suo evento terminale (es. *Closed Won* o *Closed Lost*). Il sistema calcola metriche aggregate quali media, mediana e deviazione standard per smussare l'effetto dei valori anomali (outlier).

2. **Processing Time e Waiting Time:** Scomponendo il tempo di transizione tra due nodi collegati del grafo, il sistema determina il tempo speso in ogni singola fase, evidenziando gli stati in cui le opportunità commerciali rimangono "congelate" per periodi ingiustificati rispetto alle policy di SLA (*Service Level Agreement*) aziendali.
3. **Rework Rate:** Una metrica personalizzata che identifica cicli o auto-anelli (self-loops) all'interno della medesima pratica, un chiaro indicatore di inefficienza e dispendio inutile di risorse umane.

A livello implementativo, sfruttando la libreria Polars per l'analisi vettoriale delle finestre temporali, il calcolo dei *Lead Time* risulta estremamente efficiente:

```
import polars as pl
```

```
def calculate_lead_times(df: pl.DataFrame) -> pl.DataFrame:
    # Calcolo del tempo totale di attraversamento per ogni caso
    lead_times = df.group_by("case_id").agg([
        pl.col("timestamp").min().alias("start_time"),
        pl.col("timestamp").max().alias("end_time")
    ]).with_columns(
        (pl.col("end_time") - pl.col("start_time")).dt.total_days().alias("lead_time")
    )
    return lead_times
```

```
def identify_bottlenecks(dfg_performance: dict, threshold_days: int) -> list:
    bottlenecks = []
    for edge, time_seconds in dfg_performance.items():
        time_days = time_seconds / 86400
        if time_days > threshold_days:
            bottlenecks.append({
                "source": edge[0],
                "target": edge[1],
                "delay_days": round(time_days, 2)
            })
    return bottlenecks
```

L'elenco dei *bottleneck* così individuato alimenta dinamicamente la componente visuale dell'applicazione: il diagramma a nodi e archi colora di rosso acceso (grazie a un gradiente termico associato alla scala dei tempi) le transizioni critiche, guidando istantaneamente l'occhio dell'analista verso i problemi sistemici da sanare.

Capitolo 4

ANALISI PREDITTIVA E INTEGRAZIONE AVANZATA

L'estrazione dei modelli di processo e l'individuazione dei colli di bottiglia forniscono un'eccezionale vista diagnostica sulle inefficienze storiche. Tuttavia, affinché un sistema di *Process Intelligence* generi un ritorno sull'investimento (ROI) tangibile e immediato, deve evolvere da uno strumento puramente retrospettivo a uno strumento *predittivo* e *pre-scrittivo*. Questo capitolo illustra l'integrazione di modelli di *Machine Learning* per la previsione degli esiti commerciali e l'implementazione di architetture di *Reverse ETL* per chiudere il flusso informativo, riportando il valore generato direttamente nel CRM nativo.

4.1 Feature Engineering sui log di processo

Gli algoritmi di *Machine Learning* supervisionato richiedono in input dataset tabellari in cui ogni riga rappresenta un'istanza indipendente e ogni colonna una variabile indipendente (*feature*). Un *Event Log*, per sua natura, è invece un dataset sequenziale e relazionale: un singolo *Deal* (caso) è spalmato su decine di righe temporali.

Per addestrare un modello capace di prevedere se un'opportunità commerciale ancora aperta verrà vinta (*Closed Won*) o persa (*Closed Lost*), è stato necessario sviluppare una complessa pipeline di *Feature Engineering*. L'obiettivo di questa fase è "schiacciare" la sequenza temporale di un caso in un singolo vettore di stato, estraendo indicatori sintetici ad altissimo potenziale predittivo.

Il modulo *FeatureExtractor* elabora il log calcolando per ogni pratica:

- **Feature Temporali:** Il tempo totale trascorso dalla creazione (*Lead Time*), il tempo speso nella fase corrente e l'eventuale superamento delle soglie critiche (*Bottleneck Flag*).

- **Feature Comportamentali:** Il numero totale di transizioni effettuate, il conteggio dei *rework* (ritorni a fasi precedenti) e l'identificazione di deviazioni dal percorso standard (calate dal *Conformance Checking*).
- **Feature di Contesto:** Il valore economico stimato del *Deal* (*Amount*) e la risorsa (commerciale) attualmente assegnata alla pratica.

4.2 Modelli predittivi per il rischio e il churn

Una volta ingegnerizzato il dataset di addestramento, il sistema implementa modelli di *Classificazione Binaria* avvalendosi della libreria **Scikit-learn**. A causa della natura non lineare delle interazioni tra le *feature* di processo, modelli lineari classici (come la Regressione Logistica) si sono rivelati inadeguati. Di conseguenza, l'architettura predittiva si affida ad algoritmi basati su *Ensemble Learning*, nello specifico il **Random Forest Classifier**.

Il Random Forest opera costruendo una moltitudine di alberi decisionali durante la fase di training e restituendo come previsione la moda delle classi previste dai singoli alberi. Questo approccio garantisce una notevole robustezza contro l'overfitting e gestisce nativamente variabili categoriche e valori mancanti.

Il modello viene addestrato esclusivamente sui *Deal* che hanno già raggiunto uno stato terminale (vinto o perso). Successivamente, la fase di inferenza (*Predictive Scoring*) viene applicata ai *Deal* attualmente "in volo" (in lavorazione). Il modello non si limita a prevedere l'esito finale, ma restituisce una stima probabilistica (da 0% a 100%) nota come *Win Probability Score*.

Di seguito un estratto della logica di *training* e inferenza implementata nei *worker* asincroni:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
```

```
class PredictiveService:
    def __init__(self, n_estimators=100):
        self.model = RandomForestClassifier(
            n_estimators=n_estimators,
            class_weight='balanced',
            random_state=42
        )
```

```

def train_model(self, X_train, y_train):
    self.model.fit(X_train, y_train)

def generate_predictions(self, X_live: pl.DataFrame) -> list:
    # Estrazione della probabilità di classe positiva (Win)
    probabilities = self.model.predict_proba(X_live)[: , 1]

    predictions = []
    for deal_id, prob in zip(X_live['case_id'], probabilities):
        predictions.append({
            "deal_id": deal_id,
            "win_probability": round(prob * 100, 2),
            "risk_flag": "HIGH" if prob < 0.3 else "LOW"
        })
    return predictions

```

4.3 Meccanismi di Reverse ETL

Il mero calcolo delle probabilità predittive e dei KPI di processo su una *Dashboard* esterna costringerebbe gli operatori commerciali a un continuo cambio di contesto (*Context Switching*) tra il CRM aziendale e l'applicativo analitico. Per massimizzare l'adozione della piattaforma, l'infrastruttura implementa un paradigma di **Reverse ETL**.

Il Reverse ETL è il processo architetturale mediante il quale i dati elaborati, arricchiti e modellati all'interno di un moderno *Data Stack* (in questo caso Python/DuckDB) vengono estratti e re-iniettati nei sistemi operativi (*System of Records*) da cui originariamente provenivano, come HubSpot.

4.3.1 Sincronizzazione dei KPI e degli score su HubSpot

La *task* asincrono `integration_task`, gestito tramite Celery, si occupa di orchestrare la sincronizzazione bidirezionale. Il sistema provvede innanzitutto a creare programmaticamente all'interno di HubSpot delle *Custom Properties* (Proprietà Personalizzate) associate all'oggetto *Deal*, quali ad esempio `pm_win_probability`, `pm_bottleneck_flag` e `pm_rework_count`.

Successivamente, attraverso chiamate API di tipo PATCH eseguite in *batch* per rispettare i vincoli di rete, il motore aggiorna i record sul CRM. Questo approccio arricchisce il dato grezzo nativo con gli insight algoritmici.

```
@celery_app.task(bind=True, queue='integration', max_retries=3)
```

```

def sync_predictions_to_hubspot(self, predictions_batch: list):
    client = HubSpotClient()
    success_count = 0

    for record in predictions_batch:
        deal_id = record['deal_id']
        payload = {
            "properties": {
                "pm_win_probability": record['win_probability'],
                "pm_risk_status": record['risk_flag']
            }
        }
        try:
            client.update_deal(deal_id, payload)
            success_count += 1
        except Exception as exc:
            logger.error(f"Sync fallita per il deal {deal_id}")
            raise self.retry(exc=exc, countdown=60)

    return {"status": "success", "synced_records": success_count}

```

4.3.2 Generazione di alert proattivi e workflow trigger

La presenza di queste metriche avanzate direttamente all'interno delle schede cliente di HubSpot sblocca capacità operative precedentemente irrealizzabili. Poiché i campi custom generati dal *Process Mining* sono riconosciuti nativamente dal motore di automazione di HubSpot, l'azienda può configurare *Workflow* e logiche condizionali (*Trigger*) basate sui dati algoritmici.

A titolo esemplificativo, è possibile automatizzare l'invio di un *alert* via email o su canali di messaggistica aziendale (es. Slack) al *Sales Manager* ogniqualvolta un'opportunità commerciale dal valore superiore a 10.000 Euro entri in uno stato identificato dal motore come "Collo di Bottiglia" per più di 5 giorni consecutivi, oppure se il suo *Win Probability Score* crolla improvvisamente al di sotto del 30% a seguito di una variazione anomala di processo.

Tale integrazione chiude il cerchio della *Data-Driven Enterprise*: l'analisi storica alimenta il modello matematico, il modello genera previsioni, e le previsioni attivano protocolli di salvataggio operativi prima che il danno economico si concretizzi.

Capitolo 5

PRESENTAZIONE DEI RISULTATI E UX

L'efficacia di un sistema di *Process Intelligence* non si misura esclusivamente dalla sofisticazione dei suoi algoritmi matematici, ma anche e soprattutto dal suo grado di usabilità e dalla capacità di tradurre *insight* complessi in informazioni visivamente intuitive. Per garantire l'adozione dello strumento da parte degli stakeholder aziendali, il backend analitico è stato accoppiato a un livello di presentazione (Frontend) reattivo, progettato secondo i principi della *User Experience* (UX) orientata ai dati.

5.1 Progettazione della User Interface (Streamlit)

Per lo sviluppo dell'interfaccia utente è stato impiegato **Streamlit**¹, un framework open-source basato su Python. La scelta di questa tecnologia, in alternativa a framework frontend classici come React o Angular, è motivata dalla sua integrazione nativa con l'ecosistema data-scientifico (Polars, Pandas, PM4Py, Scikit-learn). Streamlit permette di aggiornare reattivamente il *Document Object Model* (DOM) a ogni interazione dell'utente con i parametri di input, azzerando la necessità di gestire complessi stati client-side o scrivere codice boilerplate in JavaScript.

L'applicazione è stata strutturata secondo un'architettura multipagina (*Multipage App*), dividendo logicamente i domini operativi per evitare il sovraccarico cognitivo dell'utente. Il punto di ingresso (`main.py`) funge da router, caricando dinamicamente i moduli richiesti attraverso una barra di navigazione laterale:

```
import streamlit as st
```

```
def main():
```

¹Streamlit Inc., [2024](#).

```

st.set_page_config(
    page_title="Process Mining Enterprise",
    page_icon="Analytics",
    layout="wide"
)

st.sidebar.title("Navigazione")
page = st.sidebar.radio("Seleziona un modulo:", [
    "Dashboard Processo",
    "Qualità dei Dati",
    "Insights Predittivi",
    "Impostazioni di Rete"
])

if page == "Dashboard Processo":
    render_process_dashboard()
elif page == "Qualità dei Dati":
    render_data_quality()
elif page == "Insights Predittivi":
    render_predictive_insights()
else:
    render_settings()

```

5.2 Visualizzazione interattiva dei grafi e dei KPI

Il modulo centrale della piattaforma è la **Dashboard Processo**, che espone visivamente il *Directly-Follows Graph* (DFG) calcolato dal motore di *Discovery*.

Per la renderizzazione dei grafi orientati è stata utilizzata l'integrazione tra PM4Py e **Graphviz**. Poiché i grafi aziendali possono diventare estremamente densi, l'interfaccia mette a disposizione dell'analista una serie di *slider* interattivi per calibrare "al volo" le soglie algoritmiche (es. la *Dependency Threshold* dell'Heuristics Miner). Modificando questi parametri, l'utente applica un filtro euristico che fa sparire in tempo reale gli archi corrispondenti a deviazioni rare o rumore di fondo, mettendo a nudo il nucleo del processo (*Happy Path*).

A corredo della topologia di rete, la dashboard presenta i KPI di processo estratti dal KPIService. Sfruttando i componenti metrici di Streamlit (`st.metric`), vengono messi in risalto gli scostamenti prestazionali (*delta*) rispetto alle medie storiche. I colli di botti-

glia vengono esplicitati in una tabella dedicata, ordinata in modo decrescente per tempo di ritardo accumulato, permettendo al management di identificare immediatamente le fasi di *pipeline* (es. "Attesa Firma Contratto") che necessitano di un intervento manageriale.

5.3 Dashboard per il monitoraggio predittivo e della qualità dei dati

Oltre all'esplorazione diagnostica, la piattaforma fornisce due cruscotti operativi specializzati: il monitoraggio della conformità dei dati e l'intelligenza predittiva.

Il Modulo di Data Quality

Come discusso nei capitoli precedenti, l'integrità del log è garantita da **Pandera**. Il modulo "Qualità dei Dati" del frontend si collega al database analitico DuckDB e interroga la tabella degli *audit*. Questa vista offre indicatori visivi (semafori o gauge chart) che riassumono:

- La percentuale di record che hanno superato i controlli di schema.
- Il numero di anomalie temporali rilevate (es. timestamp invertiti).
- Lo stato del tasso di completamento (*Completeness Rate*) dei campi obbligatori.

In presenza di errori bloccanti, l'interfaccia espone i log dettagliati per guidare l'operatore IT nella bonifica del dato sorgente sul CRM.

Il Modulo di Predictive Insights

Il modulo "Insights Predittivi" è dedicato all'esposizione dei risultati del Machine Learning e rappresenta lo strumento operativo quotidiano per i *Sales Manager*. La vista principale è composta da una griglia interattiva che elenca tutti i *Deal* attualmente aperti (*Work in Progress*). Per ogni pratica, l'interfaccia evidenzia:

1. L'identificativo univoco della pratica e l'agente assegnatario.
2. Il **Win Probability Score** (generato dal modello Random Forest), visualizzato tramite una barra di progresso colorata (verde per probabilità > 70%, giallo, e rosso per probabilità critiche < 30%).
3. Un pulsante di *Action Trigger* che permette di forzare manualmente il processo di **Reverse ETL**, sincronizzando istantaneamente i nuovi score calcolati verso l'istanza di HubSpot senza attendere il batch asincrono programmato.

Questa struttura *user-friendly* trasforma un complesso elaboratore matematico in uno strumento di supporto alle decisioni (DSS) ad altissima accessibilità, massimizzando il ritorno pratico dell'investimento tecnologico.

Capitolo 6

CONCLUSIONI E SVILUPPI FUTURI

6.1 Valutazione dei risultati raggiunti e impatto aziendale

Il presente lavoro di tesi ha dimostrato con successo la fattibilità e l'immenso valore strategico derivante dall'applicazione del *Process Mining* ai dati transazionali di un sistema CRM. Partendo da un'infrastruttura di tracciamento puramente documentale (HubSpot), è stata ingegnerizzata una pipeline architetturale *End-to-End* in grado di estrarre, validare e modellare matematicamente le dinamiche operative della forza vendita.

L'impatto aziendale generato dall'adozione di tale sistema si è rivelato tangibile su molteplici fronti. Il management ha abbandonato un approccio decisionale basato su assunzioni teoriche per abbracciare un paradigma strettamente *Data-Driven*. La visibilità oggettiva sui veri flussi di lavoro ha permesso di individuare inefficienze sistemiche, quantificare i ritardi legati ai colli di bottiglia (*Bottlenecks*) e misurare lo scostamento tra le regole aziendali e la pratica quotidiana (*Conformance Checking*). Inoltre, l'implementazione del Machine Learning per il calcolo del *Win Probability Score* e la geniale integrazione del *Reverse ETL* hanno trasformato un sistema di reportistica passivo in uno strumento proattivo, capace di allertare gli operatori e innescare automatismi operativi direttamente nel CRM.

6.2 Analisi delle criticità riscontrate

La realizzazione di un'infrastruttura analitica così complessa ha comportato il superamento di diverse sfide ingegneristiche e metodologiche. In primo luogo, l'interfacciamento con le API cloud ha posto severi limiti operativi: la restrizione sul volume di chiamate (*Rate Limiting*) ha rischiato di compromettere l'ingestione massiva dei dati storici. Tale

criticità è stata brillantemente risolta adottando un'architettura asincrona (Celery, Redis) accoppiata a logiche di *Exponential Backoff* (Tenacity).

Una seconda problematica rilevante ha riguardato la *Data Quality*. I dati grezzi inseriti manualmente dagli operatori nel CRM presentavano inevitabilmente rumore, campi vuoti e incongruenze temporali. È stato quindi imperativo sviluppare un robusto layer di *Data Governance* tramite *Pandera*, impedendo che *garbage data* corrompessero i modelli matematici a valle. Infine, sul fronte algoritmico, si è riscontrato che l'elevata variabilità del comportamento umano produceva modelli di processo illeggibili (*Spaghetti Process*) se elaborati con algoritmi classici. L'impiego dell'*Heuristics Miner* di PM4Py ha permesso di aggirare l'ostacolo, filtrando il rumore e isolando il core business process aziendale.

6.3 Considerazioni lavorative e prospettive di sviluppo futuro

Il progetto ha posto basi ingegneristiche solide e facilmente scalabili. Guardando agli sviluppi futuri, la piattaforma si presta ad essere espansa in diverse direzioni tecnologiche:

- **Elaborazione in Streaming:** Transizione da un'estrazione *batch* asincrona a un'architettura puramente *Event-Driven*. Sfruttando i Webhook nativi di HubSpot e un broker come Apache Kafka, il sistema potrebbe aggiornare i grafi di processo e gli score predittivi in tempo reale.
- **Integrazione con Modelli Generativi (LLM):** L'interfaccia utente potrebbe essere arricchita con un assistente virtuale basato su un *Large Language Model*. Invece di esplorare manualmente la dashboard, il management potrebbe interrogare il sistema in linguaggio naturale (es. "*Qual è stata la causa principale di ritardo nel mese di Aprile?*"), demandando al backend la traduzione della domanda in query analitiche (Text-to-SQL su DuckDB) e l'estrazione degli *insight*.
- **Root Cause Analysis potenziata:** Arricchimento del modulo di Machine Learning non solo per prevedere la probabilità di successo, ma per quantificare in modo deterministico quali specifiche azioni abbiano il maggior peso nell'aumentare le probabilità di conversione, offrendo raccomandazioni prescrittive al singolo venditore.

In conclusione, il *Process Intelligence* si conferma un ecosistema vitale. Unire la flessibilità di Python all'ingegneria dei dati moderna non solo ottimizza il tempo e le risorse umane, ma posiziona l'organizzazione in un'ottica di miglioramento continuo e vantaggio competitivo.

BIBLIOGRAFIA E SITOGRAFIA

- HubSpot, Inc. (2024). *HubSpot API Developer Documentation*. URL: <https://developers.hubspot.com/docs/api/overview>.
- Van der Aalst, W. M. P. (2016). *Process Mining: Data Science in Action*. 2nd. Springer.
- Unione Europea (2016). «Regolamento (UE) 2016/679 del Parlamento europeo e del Consiglio del 27 aprile 2016 relativo alla protezione delle persone fisiche con riguardo al trattamento dei dati personali (GDPR)». In: *Gazzetta ufficiale dell'Unione europea* L 119.
- Berti, A., B. F. van Dongen e W. M. P. van der Aalst (2019). «Process Mining for Python (PM4Py): bridging the gap between process-and data science». In: *Proceedings of the ICPM Demo Track*. Vol. 2374. CEUR-WS.org, pp. 13–16.
- Bantilan, N. (2020). «Pandera: Statistical Data Validation of Pandas Dataframes». In: *Proceedings of the 19th Python in Science Conference*.
- Streamlit Inc. (2024). *Streamlit Documentation: The fastest way to build and share data apps*. URL: <https://docs.streamlit.io/>.